

UNIVERSITÉ ABDELMALEK ESSAÂDI

École Nationale des Sciences Appliquées d'Al Hoceima — ENSAH

Filière : Transformation Digitale et Intelligence Artificielle

Module : Deep Learning

Compte Rendu de TP N°1

Single Layer Perceptron (SLP)

From Scratch · Scikit-learn · Keras/TensorFlow

Étudiante

ELBALLAOUI Hasnae

Encadrant

Pr. CHERRADI Mohamed

Année universitaire

2025 / 2026

Table des matières

1. Introduction au Perceptron Simple	2
2. Exercice 1 — Portes Logiques AND et OR	3
2.1 Description du problème	3
2.2 Implémentation manuelle (From Scratch)	3
2.3 Implémentation avec Scikit-learn	4
2.4 Implémentation avec Keras	5
3. Exercice 2 — Classification binaire sur Iris Setosa	6
4. Exercice 3 — Régression linéaire via le Perceptron	9
5. Exercice 4 — Reconnaissance de chiffres MNIST	12
6. Exercice 5 — Segmentation de données synthétiques	14
7. Conclusion et perspectives	23

1. Introduction au Perceptron Simple

Le Perceptron Simple (Single Layer Perceptron, SLP) constitue l'unité fondamentale de tout réseau de neurones artificiel. Inventé par Frank Rosenblatt en 1957, il modélise de manière mathématique le comportement d'un neurone biologique. Son principe repose sur une combinaison linéaire des entrées pondérées, suivie de l'application d'une fonction d'activation qui produit une décision binaire.

L'objectif de ce travail pratique est de maîtriser le fonctionnement interne du SLP en l'implémentant selon trois niveaux d'abstraction croissants :

- From Scratch — codage manuel en NumPy pour comprendre la mécanique interne
- Scikit-learn — utilisation de la bibliothèque ML standard pour valider les résultats
- Keras / TensorFlow — exploitation d'un framework de deep learning professionnel

La règle d'apprentissage du Perceptron, dite règle delta, met à jour les poids de la façon suivante :

$$\Delta \mathbf{w}_i = \eta \cdot (\mathbf{y} - \hat{\mathbf{y}}) \cdot \mathbf{x}_i \quad \Delta \mathbf{b} = \eta \cdot (\mathbf{y} - \hat{\mathbf{y}}) \quad \hat{\mathbf{y}} = 1 \text{ si } (\mathbf{w} \cdot \mathbf{x} + \mathbf{b} \geq 0), \text{ sinon } 0$$

Cinq exercices progressifs sont présentés dans ce rapport : classification de portes logiques, classification sur le dataset Iris, régression linéaire, reconnaissance de chiffres MNIST, et segmentation de données synthétiques.

2. Exercice 1 — Portes Logiques AND et OR

2.1 Description du problème

Les portes logiques AND et OR constituent un point de départ idéal pour valider le Perceptron, car elles définissent des problèmes de classification linéairement séparables. L'objectif est d'entraîner le SLP à reproduire les tables de vérité de ces deux fonctions booléennes.

x_1	x_2	AND	OR
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	1

Tableau 1 — Table de vérité des portes AND et OR

2.2 Implémentation manuelle (From Scratch)

L'implémentation manuelle se décompose en trois fonctions : la fonction d'activation à seuil, la boucle d'entraînement avec mise à jour des poids, et la fonction de prédiction.

```
import numpy as np

# Fonction d'activation : seuil en 0
def step(z):
    return 1 if z >= 0 else 0

# Boucle d'entraînement du Perceptron
def fit_perceptron(X, y, lr=0.1, epochs=10):
    w = np.zeros(X.shape[1])
    b = 0
    for _ in range(epochs):
        for xi, yi in zip(X, y):
            y_hat = step(np.dot(w, xi) + b)
            err = yi - y_hat
            w += lr * err * xi
            b += lr * err
    return w, b

# Données AND / OR
X = np.array([[0,0],[0,1],[1,0],[1,1]])
y_and = np.array([0,0,0,1])
y_or = np.array([0,1,1,1])

w_and, b_and = fit_perceptron(X, y_and)
w_or, b_or = fit_perceptron(X, y_or)
```

Résultat — AND : [0, 0, 0, 1] | OR : [0, 1, 1, 1] ✓Convergence parfaite en quelques époques.

2.3 Implémentation avec Scikit-learn

La classe Perceptron de Scikit-learn encapsule la même logique en proposant une interface unifiée fit / predict, standard dans l'écosystème Python ML.

```

from sklearn.linear_model import Perceptron
import numpy as np

X      = np.array([[0,0],[0,1],[1,0],[1,1]])
y_and = np.array([0,0,0,1])
y_or  = np.array([0,1,1,1])

# Porte AND
clf_and = Perceptron(eta0=0.1, random_state=42)
clf_and.fit(X, y_and)
print('AND :', clf_and.predict(X))

# Porte OR
clf_or = Perceptron(eta0=0.1, random_state=42)
clf_or.fit(X, y_or)
print('OR  :', clf_or.predict(X))

```

Résultat Scikit-learn — AND : [0 0 0 1] | OR : [0 1 1 1] ✓Identique à l'implémentation manuelle.

2.4 Implémentation avec Keras

Avec Keras, le Perceptron se traduit par un modèle séquentiel doté d'une unique couche Dense(1) et d'une activation sigmoïde (approximation différentiable de la fonction échelon), compilé avec une perte binaire.

```

from tensorflow import keras
import numpy as np

X      = np.array([[0,0],[0,1],[1,0],[1,1]], dtype=float)
y_and = np.array([0,0,0,1], dtype=float)
y_or  = np.array([0,1,1,1], dtype=float)

def build_slp():
    m = keras.Sequential([
        keras.layers.Dense(1, activation='sigmoid', input_shape=(2,))
    ])
    m.compile(optimizer=keras.optimizers.Adam(lr=0.1),
              loss='binary_crossentropy', metrics=['accuracy'])
    return m

# Entraînement AND
slp_and = build_slp()
slp_and.fit(X, y_and, epochs=1000, verbose=0)
pred = (slp_and.predict(X) > 0.5).astype(int).flatten()
print('AND Keras :', pred)

```

Analyse et commentaires

Les deux portes logiques sont linéairement séparables : il existe un hyperplan (ici une droite dans $\mathbb{R}^2$) capable de partitionner correctement les sorties. Le SLP converge sans difficulté pour AND et OR, quelle que soit l'implémentation. La porte XOR, en revanche, nécessiterait un réseau multicouche (MLP), car elle n'est pas linéairement séparable.

3. Exercice 2 — Classification binaire sur Iris Setosa

3.1 Description

Le dataset Iris contient 150 observations de trois espèces : Setosa, Versicolor et Virginica. La tâche consiste à entraîner le Perceptron à distinguer Iris Setosa (label 1) des deux autres espèces (label 0) en utilisant uniquement la longueur et la largeur des pétales (colonnes 2 et 3). Iris Setosa étant linéairement séparable dans cet espace de caractéristiques, le SLP devrait atteindre une précision proche de 100 %.

3.2 Implémentation From Scratch

```
import numpy as np
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split

iris = load_iris()
X = iris.data[:, 2:4]          # longueur & largeur pétale
y = (iris.target == 0).astype(int) # 1=Setosa, 0=autres

X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.2, random_state=0)

def step(z): return 1 if z >= 0 else 0

def train(X, y, lr=0.01, epochs=20):
    w, b = np.zeros(X.shape[1]), 0
    for _ in range(epochs):
        for xi, yi in zip(X, y):
            e = yi - step(np.dot(w, xi) + b)
            w += lr * e * xi
            b += lr * e
    return w, b

w, b = train(X_tr, y_tr)
preds = np.array([step(np.dot(xi, w) + b) for xi in X_te])
print(f'Accuracy From Scratch : {np.mean(preds == y_te):.2f}')
```

Résultat — Accuracy : 1.00 (100 %) ✓ Frontière de décision linéaire parfaite sur les pétales.

3.3 Implémentation Scikit-learn

```
from sklearn.linear_model import Perceptron
from sklearn.metrics import accuracy_score, classification_report

clf = Perceptron(random_state=42)
clf.fit(X_tr, y_tr)
y_pred = clf.predict(X_te)
print(f'Accuracy Scikit-learn : {accuracy_score(y_te, y_pred):.4f}')
print(classification_report(y_te, y_pred))
```

3.4 Implémentation Keras

```
from sklearn.preprocessing import StandardScaler
from tensorflow import keras

scaler = StandardScaler()
X_tr_n = scaler.fit_transform(X_tr)
X_te_n = scaler.transform(X_te)

model = keras.Sequential([
```

```
keras.layers.Dense(1, activation='sigmoid', input_shape=(2,))
])
model.compile(optimizer=keras.optimizers.Adam(0.01),
              loss='binary_crossentropy', metrics=['accuracy'])
model.fit(X_tr_n, y_tr, epochs=200, verbose=0)

acc = model.evaluate(X_te_n, y_te, verbose=0)[1]
print(f'Accuracy Keras : {acc:.4f}')
```

Analyse et commentaires

Iris Setosa est parfaitement isolable des deux autres espèces dans l'espace pétale (longueur × largeur). Les trois implémentations aboutissent à une précision maximale. La normalisation StandardScaler utilisée dans la version Keras accélère la descente de gradient en homogénéisant les plages de valeurs des deux features.

4. Exercice 3 — Régression linéaire via le Perceptron

4.1 Description

Le but de cet exercice est d'adapter le SLP à un problème de régression. On génère 100 points suivant la relation $y = 2x + 1$ avec un bruit gaussien d'écart-type 0,5, sur l'intervalle $[0, 10]$. En supprimant la fonction d'activation à seuil (activation linéaire), le Perceptron se comporte comme une régression par descente de gradient stochastique. Les poids cibles à apprendre sont $w \approx 2$ et $b \approx 1$.

4.2 Implémentation From Scratch

```
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error

np.random.seed(42)
X = np.linspace(0, 10, 100)
y = 2 * X + 1 + np.random.randn(100) * 0.5

X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.2, random_state=42)

class LinearPerceptron:
    def __init__(self, lr=0.001, epochs=1000):
        self.lr, self.epochs = lr, epochs
        self.w = self.b = 0.0

    def predict(self, x): return self.w * x + self.b

    def fit(self, X, y):
        for _ in range(self.epochs):
            for xi, yi in zip(X, y):
                err = yi - self.predict(xi)
                self.w += self.lr * err * xi
                self.b += self.lr * err
            return self

reg = LinearPerceptron().fit(X_tr, y_tr)
mse = mean_squared_error(y_te, reg.predict(X_te))
print(f'MSE : {mse:.4f} | w={reg.w:.4f} | b={reg.b:.4f}')
```

Résultat — MSE ≈ 0.1544 | $w \approx 2.008$ | $b \approx 0.914$ ✓ Très proche des valeurs théoriques ($w=2$, $b=1$).

4.3 Implémentation Scikit-learn

```
from sklearn.linear_model import SGDRegressor

sgd = SGDRegressor(max_iter=1000, learning_rate='constant',
                    eta0=0.001, random_state=42)
sgd.fit(X_tr.reshape(-1,1), y_tr)
mse_sk = mean_squared_error(y_te, sgd.predict(X_te.reshape(-1,1)))
print(f'Scikit-learn MSE : {mse_sk:.4f}')
```

SGDRegressor est l'équivalent direct du Perceptron de régression : il minimise le MSE par descente de gradient stochastique, sans fonction d'activation non-linéaire.

4.4 Implémentation Keras

```
from tensorflow import keras

model = keras.Sequential([
    keras.layers.Dense(1, activation='linear', input_shape=(1,))
])
model.compile(optimizer=keras.optimizers.Adam(0.1), loss='mse')
model.fit(X_tr.reshape(-1,1), y_tr, epochs=200, verbose=0)

mse_k = mean_squared_error(y_te, model.predict(X_te.reshape(-1,1)).flatten())
print(f'Keras MSE : {mse_k:.4f}')
```

Analyse et commentaires

En désactivant la non-linéarité (`activation='linear'`), le réseau de neurones se réduit strictement à une régression linéaire. Les trois approches convergent vers des valeurs de w très proches de 2 et b proche de 1, confirmant que le SLP est capable d'apprendre une relation affine simple. Un MSE inférieur à 0,3 valide l'apprentissage.

5. Exercice 4 — Reconnaissance de chiffres MNIST

5.1 Description

MNIST est un benchmark classique du deep learning composé de 70 000 images de chiffres manuscrits (0–9) en niveaux de gris 28×28 pixels (784 features). Pour limiter le temps de calcul, on travaille sur un sous-ensemble de 5 000 images. La nature multi-classes du problème (10 classes) impose l'utilisation d'une stratégie One-vs-All pour les perceptrons binaires.

5.2 Implémentation From Scratch — One-vs-All

```
import numpy as np
from sklearn.datasets import fetch_openml
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

mnist = fetch_openml('mnist_784', version=1, as_frame=False)
X, y = mnist.data[:5000] / 255.0, mnist.target[:5000].astype(int)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.2, random_state=42)

class OvAPerceptron:
    def __init__(self, lr=0.01, n_iter=10):
        self.lr=lr; self.n_iter=n_iter

    def fit(self, X, y, n_cls=10):
        n, d = X.shape
        self.W = np.zeros((n_cls, d)); self.b = np.zeros(n_cls)
        for _ in range(self.n_iter):
            for xi, yi in zip(X, y):
                scores = self.W @ xi + self.b
                pred = scores.argmax()
                if pred != yi:
                    self.W[yi] += self.lr * xi; self.b[yi] += self.lr
                    self.W[pred] -= self.lr * xi; self.b[pred] -= self.lr

    def predict(self, X):
        return (X @ self.W.T + self.b).argmax(axis=1)

clf = OvAPerceptron(); clf.fit(X_tr, y_tr)
print(f'OvA Accuracy : {accuracy_score(y_te, clf.predict(X_te)):.4f}')
```

5.3 Scikit-learn et Keras

```
# - Scikit-learn -----
from sklearn.linear_model import Perceptron
sk = Perceptron(max_iter=50, random_state=42)
sk.fit(X_tr, y_tr)
print(f'Sk-learn : {accuracy_score(y_te, sk.predict(X_te)):.4f}')
```

```
# - Keras -----
from tensorflow import keras
model = keras.Sequential([
    keras.layers.Dense(10, activation='softmax', input_shape=(784,))
])
model.compile(optimizer=keras.optimizers.Adam(0.01),
              loss='sparse_categorical_crossentropy', metrics=['accuracy'])
model.fit(X_tr, y_tr, epochs=20, verbose=0)
_, acc = model.evaluate(X_te, y_te, verbose=0)
print(f'Keras : {acc:.4f}')
```

Analyse et commentaires

Le SLP atteint une précision raisonnable sur MNIST (~80–85 %) mais souffre de sa limite fondamentale : la frontière de décision linéaire. Les confusions sont fréquentes entre chiffres de forme similaire (4/9, 3/8, 5/6). La stratégie One-vs-All étend efficacement le modèle aux 10 classes. Keras, grâce à softmax, gère naturellement la classification multi-classes. Un CNN obtiendrait un résultat supérieur à 99 % sur ce jeu de données.

6. Exercice 5 — Segmentation de données synthétiques

6.1 Description

L'objectif est de générer un jeu de données synthétiques à l'aide de `make_blobs` (300 points, 3 centres, `cluster_std=1.2`), d'entraîner un Perceptron binaire sur deux des trois clusters (approche supervisée), puis de comparer les résultats avec l'algorithme KMeans (approche non supervisée opérant sur les 3 clusters).

6.2 Génération et visualisation des données

```
from sklearn.datasets import make_blobs
import matplotlib.pyplot as plt

X, y = make_blobs(n_samples=300, centers=3, cluster_std=1.2, random_state=42)

couleurs = ['crimson', 'royalblue', 'forestgreen']
for i in range(3):
    plt.scatter(X[y==i, 0], X[y==i, 1], c=couleurs[i],
                edgecolors='k', label=f'Groupe {i}')
plt.title('Données synthétiques - 3 groupes')
plt.legend(); plt.tight_layout(); plt.show()
```

6.3 Implémentation From Scratch (classification binaire groupes 0 et 1)

```
from sklearn.preprocessing import StandardScaler

# Sélection des 2 premiers groupes
mask = y != 2
X_bin, y_bin = X[mask], y[mask]

scaler = StandardScaler()
X_sc = scaler.fit_transform(X_bin)
X_tr, X_te, y_tr, y_te = train_test_split(X_sc, y_bin, test_size=0.2, random_state=42)

class SLP:
    def __init__(self, lr=0.01, epochs=50):
        self.lr, self.epochs = lr, epochs
        self.w = self.b = None
        self.history = []

    def _step(self, z): return np.where(z >= 0, 1, 0)

    def fit(self, X, y):
        self.w = np.zeros(X.shape[1]); self.b = 0
        for _ in range(self.epochs):
            errs = 0
            for xi, yi in zip(X, y):
                delta = yi - self._step(self.w @ xi + self.b)
                self.w += self.lr * delta * xi
                self.b += self.lr * delta
                errs += int(delta != 0)
            self.history.append(errs)

    def predict(self, X): return self._step(X @ self.w + self.b)
    def score(self, X, y): return np.mean(self.predict(X) == y)

slp = SLP(); slp.fit(X_tr, y_tr)
print(f'[From Scratch] Accuracy : {slp.score(X_te, y_te)*100:.2f}%')
```

6.4 Implémentation Scikit-learn + KMeans

```

from sklearn.linear_model import Perceptron
from sklearn.cluster import KMeans
from sklearn.metrics import classification_report, accuracy_score

# - Perceptron supervisé -----
clf = Perceptron(max_iter=100, eta0=0.01, random_state=42)
clf.fit(X_tr, y_tr)
y_pred = clf.predict(X_te)
print('[Scikit] Rapport :')
print(classification_report(y_te, y_pred))

# - KMeans non supervisé (3 clusters) -----
X_all_sc = StandardScaler().fit_transform(X)
km = KMeans(n_clusters=3, random_state=42, n_init=10)
km.fit(X_all_sc)
labels_km = km.labels_
print('Centres KMeans :', km.cluster_centers_)

```

6.5 Implémentation Keras

```

from tensorflow import keras

model = keras.Sequential([
    keras.layers.Input(shape=(2,)),
    keras.layers.Dense(1, activation='sigmoid')
])
model.compile(optimizer=keras.optimizers.SGD(0.01),
              loss='binary_crossentropy', metrics=['accuracy'])

history = model.fit(X_tr, y_tr, epochs=100, batch_size=16,
                   validation_split=0.2, verbose=0)

loss, acc = model.evaluate(X_te, y_te, verbose=0)
print(f'[Keras] Loss={loss:.4f} | Accuracy={acc*100:.2f}%')

```

6.6 Analyse comparative

Critère	From Scratch	Scikit-learn	Keras
Précision (accuracy)	100 %	100 %	100 %
Mode de convergence	Manuel, itératif	Automatique	Automatique + validation
Extensibilité vers MLP	Faible	Moyenne	Très élevée
Transparence algorithmique	Maximale	Limitée	Limitée

Tableau 2 — Comparaison des trois implémentations sur l'Exercice 5

Critère	Perceptron (supervisé)	KMeans (non supervisé)
Besoin de labels	Oui — nécessaire à l'entraînement	Non — apprentissage autonome
Frontière de décision	Hyperplan linéaire	Régions de Voronoï
Nombre de classes	Binaire (0 et 1)	k quelconque (ici k=3)
Objectif d'optimisation	Minimiser les erreurs de classification	Minimiser l'inertie intra-cluster

Tableau 3 — Perceptron supervisé vs KMeans non supervisé

Analyse et commentaires

From Scratch : l'implémentation transparente révèle chaque étape du mécanisme : initialisation, calcul du score, décision, mise à jour. La courbe d'erreurs par époque illustre la convergence rapide (convergence en moins de 5 époques pour des clusters bien séparés).

Scikit-learn : le `classification_report` expose precision, recall et F1-score par classe, outils incontournables pour évaluer un classificateur binaire.

Keras : la couche `Dense(1, sigmoid)` émule fidèlement le SLP classique, tout en ouvrant la voie vers des architectures profondes (MLP, CNN) par simple ajout de couches.

KMeans vs Perceptron : distinction fondamentale entre apprentissage supervisé (labels requis, minimisation d'une fonction de coût supervisée) et non supervisé (découverte autonome de la structure, minimisation de l'inertie intra-groupe).

7. Conclusion et perspectives

Ce travail pratique a permis de développer une compréhension approfondie du Perceptron Simple à travers cinq exercices aux problématiques variées. En parcourant trois niveaux d'implémentation pour chacun, nous avons pu observer les mêmes algorithmes à différents degrés d'abstraction.

Les principaux enseignements sont les suivants :

1. Portes logiques (Ex. 1) : validation sur un cas trivial et linéairement séparable. Le SLP converge parfaitement sur AND et OR ; il échouerait sur XOR.
2. Classification Iris (Ex. 2) : excellente précision (100 %) grâce à la séparabilité naturelle d'Iris Setosa dans l'espace pétale. La normalisation des données accélère la convergence sous Keras.
3. Régression linéaire (Ex. 3) : en mode linéaire, le Perceptron apprend correctement $y=2x+1$ avec un MSE très faible et des poids convergent vers les valeurs théoriques.
4. MNIST (Ex. 4) : performances correctes mais plafonnées par la linéarité du modèle. La stratégie One-vs-All généralise efficacement le SLP aux 10 classes.
5. Segmentation (Ex. 5) : comparaison instructive entre Perceptron supervisé et KMeans non supervisé, mettant en lumière les fondements conceptuels distincts des deux familles d'apprentissage.

⚠ Limite fondamentale du SLP

Le Perceptron Simple ne peut modéliser que des frontières de décision linéaires. Face à des problèmes non séparables linéairement (XOR, spirales, images naturelles), il est nécessaire d'utiliser un réseau multicouche (MLP) équipé de fonctions d'activation non linéaires telles que ReLU, tanh ou GELU, et entraîné par rétropropagation du gradient.

Ce TP constitue une base solide pour aborder les réseaux de neurones profonds dans les prochains travaux pratiques du module Deep Learning.